NOMILA-V2 · PHASE A · STEP 1 · DESIGN PROPOSAL FOR REVIEW

Step 1 Social Substrate

The social intelligence data layer of spec §22, §26 and §28: identities, sync runs, sanitized raw payloads, durable content, metric observations, audience distributions and versioned features. Current → target, with every existing code path dispositioned. Additive and shadow: OAuth, fetch, display analytics and the v7 score keep running unchanged while the new pipeline writes beside them. No implementation until approved.

base: main @ Step 0 merged (8371b26) spec: §22.3, §26, §28.1–28.10, §29–31 next migration: 0044

flag: SOCIAL_SUBSTRATE_WRITE (off by default)

KEEP MODIFY EXTRACT REPLACE ADD

THE SHAPE

Six new tables, one reused, one flag, zero behaviour change

The daily snapshot, the Growth tab, the media library and the v7 score are untouched. Every fetch that already runs gets a second, sanitized destination.

WHAT THE DATA SAID

Secrets are already in stored payloads

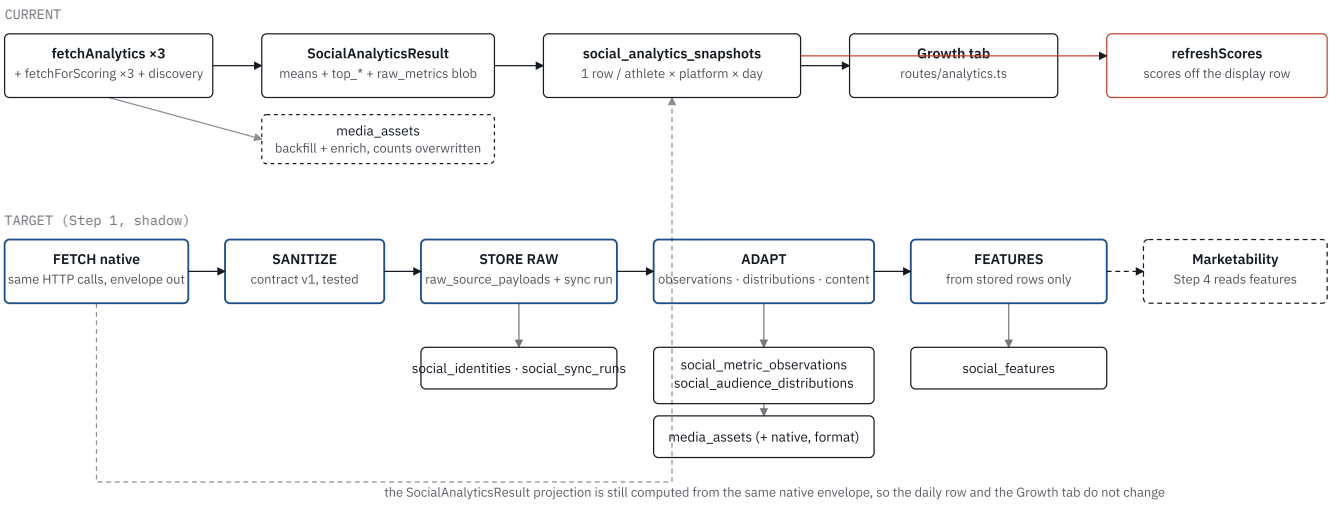
82 of 218 snapshot rows carry a token string in `raw_metrics`: 9 Instagram account-insight paging URLs with our access token, 73 TikTok cover URLs with a CDN `refresh_token` parameter. The sanitizer and a one-off scrub are in scope (section 3).

THE ONE ARCHITECTURAL CALL

Reuse media_assets as the durable content entity

Recommended with three additive columns and one rule change (counts stop being overwritten). The trade-off and the alternative are laid out in section 8 for your decision.

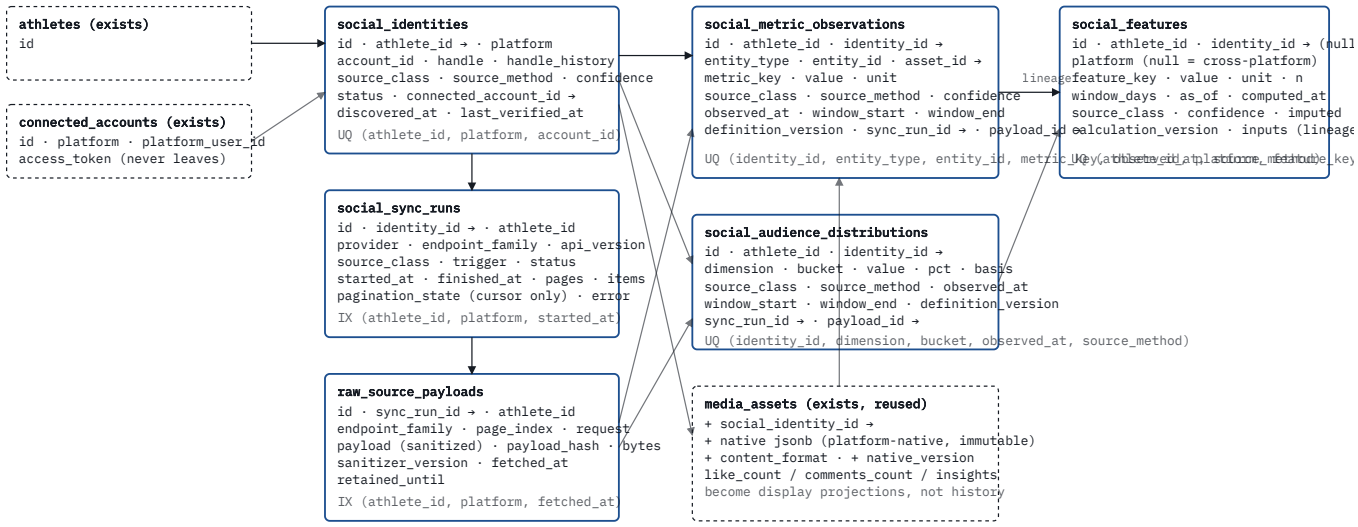
0 · The pipeline, current and target



Top: today one fetch produces a pre-averaged display row that both the Growth tab and the score read. Bottom: the same fetch keeps its HTTP calls, returns the native envelope, and that envelope is sanitized, stored, adapted into observations and distributions, and turned into versioned features. The display projection is derived from the same envelope, so nothing the athlete sees changes. Scoring moves to features in Step 4, not here.

1 · Canonical storage model

Seven logical entities from §28.3. Six are new tables; the seventh, durable content, reuses `media_assets` (section 8). All are athlete-scoped, additive, and written only behind the flag. Conventions shared by every table: `source_class` from the five-value `SourceClass`; `source_method` names the exact provider path; `observed_at` is the provider's or the fetch's instant; `window_start` / `window_end` are present only for windowed metrics; nothing is ever updated in place except the explicitly mutable status and pagination fields.



Arrows point from the referencing table to the referenced one. Identity is the spine: every observation, distribution and content row names the account it belongs to; every stored fact names the sync run and sanitized payload it came from; every feature names the observations it was computed from.

social_identities

Purpose. One row per platform account an athlete has, is or was linked to. Today the only account identity is `connected_accounts.platform_user_id`, which is overwritten per connection row (Mia has five revoked Instagram rows across two accounts, one of them the shared `nomila.co` discovery account) and typed handles in `athletes.socials` with no id at all.

COLUMN	TYPE	NOTES
`id`	uuid pk	
`athlete_id`	uuid → athletes, cascade	
`platform`	text CHECK instagram | tiktok | youtube | x	x reserved (spec §4 asks to design for four)
`account_id`	text NOT NULL	Platform-native, stable: IG user id, TikTok open_id, YouTube channel id. The identity key.
`handle · canonical_url`	text	Current handle; updated in place, prior values appended to `handle_history`
`handle_history`	jsonb [{handle, seenAt}]	Preserves prior handles (§28.3)
`source_class · source_method`	text	connected_verified / oauth_instagram_login · platform_public / meta_business_discovery · web_researched / research_extraction or athlete_typed
`confidence`	numeric(3,2)	1.0 OAuth; 0.7 discovery match on the athlete's typed handle; research per evidence
`status`	text CHECK active | superseded | unverified | rejected	A new account_id for the same platform supersedes the old row; the old row and its observations stay
`connected_account_id`	uuid → connected_accounts, set null	The credential row that proved it; a disconnect nulls nothing here (see decision D5)
`discovered_at · last_verified_at`	timestamptz	
`evidence`	jsonb	For web_researched rows: research snapshot id and the extracted handle evidence

Uniqueness (athlete_id, platform, account_id) · **Indexes** (athlete_id, platform, status) · **Retention** forever

social_sync_runs

Purpose. One row per provider sync of one identity through one endpoint family: the audit unit every payload and observation points back to. Today the only trace of a fetch is `toolsCalled[]` inside the snapshot and a log line.

COLUMN	TYPE	NOTES
`id · athlete_id · identity_id →`	uuid	
`provider · endpoint_family · api_version`	text	e.g. meta_graph / ig_media_insights / v24.0; tiktok_display / video_list / v2; youtube / data_videos or analytics_reports / v3; meta_graph / business_discovery
`source_class`	text	connected_verified or platform_public
`trigger`	text CHECK connect | nightly | backfill | discovery | manual | fixture	
`status`	text CHECK running | ok | partial | failed	partial = some pages or sub-calls failed; the run still counts
`started_at · finished_at · pages · items`		
`pagination_state`	jsonb	Opaque cursor / pageToken / numeric cursor ONLY; never a paging URL (Graph paging URLs embed the token)
`error`	jsonb	Sanitized provider error (code, subcode, message); never the request URL
`connected_account_id`	uuid → set null	

Indexes (athlete_id, platform, started_at desc) · (identity_id, endpoint_family, started_at desc) · **Retention** forever (small rows)

raw_source_payloads

Purpose. The sanitized provider response, page by page, sufficient to re-adapt later without refetching (§28.3, §28.10). Today `raw_metrics` is "the closest retained representation" (§29.1) but is already reshaped, carries secrets, and is copied whole into every daily row.

COLUMN	TYPE	NOTES
`id · sync_run_id → (cascade) · athlete_id · platform`		A payload cannot outlive its run
`endpoint_family · page_index`	text · int	
`request`	jsonb	Path, non-secret params (fields, metric, period, limit, cursor), never the token or key
`payload`	jsonb	The response envelope after the sanitizer (section 3). Full envelope by default (decision D3)
`payload_hash · bytes`	text · int	sha256 of canonical JSON; dedupes identical re-fetches
`sanitizer_version`	text NOT NULL	Every row says which contract cleaned it
`fetched_at · retained_until`	timestamptz	retained_until from the retention parameter (decision D2)

Indexes (athlete_id, platform, fetched_at desc) · (sync_run_id) · **Retention** a parameter; observations derived from a payload survive its deletion

Durable content: media_assets, reused

Analysis and trade-off in section 8. Additive columns: `social_identity_id` (uuid → social_identities), `native` (jsonb, the platform-native immutable fields the adapter preserves: TikTok title, video_description, duration, cover url stripped of query; Instagram media_product_type, is_shared_to_feed, children ids; YouTube title, description, duration, topic categories, definition), `content_format` (text, canonical: post | reel | story | carousel | carousel_frame | video | short | long_form), `native_version` (text, the adapter version that wrote `native`), `first_seen_at`. Rule

change: `like_count`, `comments_count` and `insights` keep being refreshed for display, but history lives in observations and the feature calculator never reads these columns.

social_metric_observations

Purpose. The canonical fact table of §28.4: one row per metric per entity per observation instant per source. Account-level facts today live as flat columns on the daily row and are lost when the row is overwritten by a same-day re-run; content-level facts live in `media_assets.insights` and are overwritten on every re-see.

COLUMN	TYPE	NOTES
`id`	bigint identity pk	Volume table; a uuid buys nothing here
`athlete_id · platform · identity_id →`		
`entity_type`	text CHECK account | content | audience	
`entity_id`	text	account_id for account; source_media_id for content; 'audience' for audience-level metrics
`asset_id`	uuid → media_assets, set null	Set when the content row exists; observations may precede the asset (a Story) or outlive a deleted asset
`metric_key · value · unit`	text · numeric · text	From the metric registry (section 4); unit is count | ratio | seconds | percent
`source_class · source_method · confidence`		
`observed_at`	timestamptz NOT NULL	The provider's timestamp when it gives one (Analytics report end date), else the fetch instant
`window_start · window_end`	timestamptz NULL	Only for windowed metrics (IG account reach "day", YouTube 30-day totals, demographics last_30_days). Point-in-time counts leave both null
`definition_version`	text NOT NULL	The metric definition that produced it; a provider change bumps the version, never rewrites rows
`sync_run_id → · payload_id →`	set null	Lineage to the run and the sanitized page

Uniqueness (identity_id, entity_type, entity_id, metric_key, observed_at, source_method) so re-adapting the same payload is idempotent · **Indexes** (identity_id, metric_key, observed_at desc) · (asset_id, metric_key, observed_at desc) · **Retention** forever; never updated

social_audience_distributions

Purpose. Full breakdowns (§26.4), not the top bucket. Today they exist only inside `raw_metrics.demographics` and `raw_metrics.youtube` and are unreachable by SQL.

COLUMN	TYPE	NOTES
`id · athlete_id · platform · identity_id →`		
`dimension`	text CHECK country | city | age | gender | age_gender | traffic_source | device	age_gender keeps YouTube's joint distribution rather than splitting it and losing the cross
`bucket · value · pct · basis`	text · numeric · numeric · text	basis: followers (IG follower_demographics) | viewers (YouTube viewerPercentage) | views (YouTube country/traffic/device). Not comparable across bases and never merged
`source_class · source_method · observed_at · window_start · window_end · definition_version · sync_run_id → · payload_id →`		IG demographics are lifetime-with-last_30_days timeframe; YouTube are 30-day reports

Uniqueness (identity_id, dimension, bucket, observed_at, source_method) · **Retention** forever

social_features

Purpose. Versioned, derived numbers the engines read (§28.3, §28.6). Computed from observations and distributions only; never from a provider call, never from the daily display row.

COLUMN	TYPE	NOTES

id · athlete_id		
platform · identity_id →	nullable	NULL platform = cross-platform feature (e.g. effective reach)
feature_key · value · unit · n		n = sample size behind the number (posts, days)
window_days · as_of	int · timestamptz	The feature is "the last window_days ending at as_of"
source_class · confidence · imputed		source_class = the LOWEST authority among the inputs used; imputed is always false in Step 1 (priors are Step 4)
calculation_version	text	The calculator version (section 5)
inputs	jsonb	Lineage: observation id ranges or the exact list, counts, and the resolution decisions (which source won per metric)
computed_at	timestamptz	
Uniqueness (athlete_id, coalesce(platform,'*'), feature_key, window_days, as_of, calculation_version) · **Retention** forever; a recompute under a new version adds rows		

---

2 · Platform adapter architecture

The five stages, and where each existing function lands. The rule: the HTTP calls stay where they are and are not re-implemented; what changes is what they return (a native envelope instead of a pre-averaged result) and where it goes.

STAGE	INSTAGRAM	TIKTOK	YOUTUBE	DISCOVERY (PUBLIC)
FETCH raw native	EXTRACT the Graph calls in `fetchAnalytics` (profile, account insights, media + insights, four demographic breakdowns) and `listMediaPage`, `fetchMediaInsights` into `fetchInstagramNative()` returning `{ profile, accountInsights, mediaPages[], demographics{} }` untouched	EXTRACT user/info + video/list (+ dormant creator/info) into `fetchTikTokNative()`; keep the 20-per-page cursor	EXTRACT channels, analytics reports (totals, age×gender, country, traffic, device), search+videos (or uploads playlist) into `fetchYouTubeNative()`; keep Analytics and Data responses separate	EXTRACT Business Discovery and public Data API calls into the same shape with source_class platform_public
SANITIZE	ADD `server/lib/social/sanitize.ts`, one function for every provider, applied to the envelope before it touches a table (section 3)			
STORE RAW	ADD `server/services/social/ingest.ts`: opens a sync run, stores one payload row per page, closes the run with status and counts			
ADAPT to canonical	ADD `adapters/instagram.ts`: account observations (followers, following, media_count, account_reach_day, account_views_day, profile_link_taps_day), content upserts into media_assets with `native`, content observations (like_count, comments_count, reach, saved, shares, total_interactions, video_views, ig_reels_avg_watch_time), follower distributions; the "discovery reel" rule becomes an `inferred` content feature, not a fact	ADD `adapters/tiktok.ts`: account (follower_count, following_count, likes_count, video_count, is_verified), content (view_count, like_count, comment_count, share_count) with duration, title and description kept in `native`; no reach, no saves: those keys are simply absent	ADD `adapters/youtube.ts`: account (subscriber_count, view_count_total, video_count), 30-day windowed account metrics (views_30d, minutes_watched_30d, avg_view_pct_30d, subs_gained_30d, subs_lost_30d), content (view_count, like_count, comment_count) with duration and topics in `native`; viewer distributions with basis viewers/views	Same adapters, source_class platform_public, fewer metrics
CALCULATE features	ADD `server/services/intelligence/social/features.ts`: pure, takes observation and distribution rows plus as_of and windows, returns feature rows with lineage (section 5)			

Every existing code path, dispositioned

CODE	DISPOSITION	WHAT HAPPENS IN STEP 1
`integrations/instagram.ts handleCallback · refreshTokenIfNeeded · sendDM · syncRecent`	KEEP	Untouched
`instagram.ts fetchAnalytics`	EXTRACT	Becomes `fetchInstagramNative` + `projectInstagramDisplay(native)`. The projection reproduces today's SocialAnalyticsResult byte-for-byte (pinned by a fixture test), so the daily row is unchanged

`instagram.ts fetchForScoring · tiktok.ts fetchForScoring · youtube.ts fetchForScoring · skills/fetchSocialData.ts`	KEEP now, REPLACE in Step 4	Still runs at connect and still scores. Not touched in Step 1 so the v7 path is provably unchanged. Retired when the engine reads features
`instagram.ts listMediaPage · tiktok.ts listMediaPage · youtube.ts listMediaPage`	MODIFY	Return the native page and the cursor; the backfill stores each page as a raw payload and emits content observations at fetch time (like/comment/view counts as of that page)
`instagram.ts fetchMediaInsights · skills/enrichTopPostInsights.ts`	MODIFY	Each enrichment writes reach/saved/shares/views/watch-time observations with observed_at, then updates `insights` for display as today
`integrations/discovery.ts · lib/platformDiscovery.ts`	EXTRACT	Same calls; envelope stored with source_class platform_public; the discovery row on the daily table continues (dual-write)
`skills/collectSocialAnalytics.ts · collectDiscoveryAnalytics.ts`	MODIFY	Call the native fetch once; write the display row from the projection (unchanged) AND, when the flag is on, run sanitize → store → adapt → features
`skills/mediaBackfill.ts · rehostAthleteMedia.ts ingestOne`	MODIFY	ingestOne also writes `native`, `content_format`, `social_identity_id`; pixels and enrichment untouched
`skills/refreshScores.ts buildSocialFromAnalytics · pickBestPerPlatform`	KEEP now, EXTRACT in Step 4	Untouched in Step 1. Its precedence rule is re-implemented at metric level in the feature calculator (section 6) and compared against it in the Mia validation
`integrations/types.ts SocialAnalyticsResult · SocialAnalyticsRaw`	KEEP	Remain the display contract. Marked in a header comment as "projection, not a scoring input"
`routes/analytics.ts · client Growth tab · Mila get_analytics · deck audience card`	KEEP	Read the daily row as today (section 7)
`social_analytics_snapshots.raw_metrics.recent_media daily copy`	MODIFY later	Kept in Step 1 (the projection still writes it). Stops once the display readers of recent_media are pointed at media_assets (a follow-up, not Step 1)
`tiktok.ts DEMOGRAPHICS_ENABLED / creator_info`	KEEP dormant	§28.5
`Instagram Stories`	ADD behind its own flag	A `stories` endpoint family (GET /{ig}/stories + per-story insights) with a capture cadence (decision D4); Story observations are content observations with content_format story and no media_assets row unless we choose to keep covers

3 · Secret sanitization

Verified against the live dev database on 10 Sep: **82 of 218** daily rows already hold a token string inside `raw_metrics`. Every place a secret can appear, with what was found:

LOCATION	PROVIDER	WHAT APPEARS	FOUND IN PROD DATA
Graph paging URLs `paging.next` / `paging.previous`	Instagram (account insights, media, demographics, business discovery)	Our OAuth access token as a query parameter	YES 9 rows, `raw_metrics.account_insights.paging.next`
CDN media URLs with signed query strings	TikTok `cover_image_url`; Instagram `media_url` / `thumbnail_url`	Provider CDN signature params, including one literally named `refresh_token` on TikTok covers. Not our credential, but indistinguishable by name and useless after expiry	YES 73 rows, `recent_media[].thumbnail_url`
Request query strings	All Graph calls (`access_token=`), YouTube public (`key=`), Business Discovery (`access_token=`)	Token or API key	Not persisted today; would be if `request` were stored naively
Request headers	TikTok, YouTube connected (`Authorization: Bearer`)	Token	Not persisted; log lines must never print headers
Provider error objects	Graph errors can echo the request; TikTok errors include `log_id`	Possibly the URL	Not observed; covered by the same walk

Accidental spread of a `connected_accounts` row	Any adapter	`access_token`, `refresh_token` columns	Not observed; the type system will forbid the row type in envelopes
Pagination cursors	IG `after`, YouTube `pageToken`, TikTok numeric	Opaque, no secret	Safe to persist in `pagination_state`; paging URLs are not

The contract (sanitizer v1)

1. **Exact-value redaction first.** The sanitizer receives the set of live secrets for the call (the connection's access and refresh tokens, the discovery token, the YouTube API key) and replaces any string containing one of them with `<redacted:secret>`, everywhere in the tree. This catches a token wherever it lands, including inside a URL or an error message.
2. **Key-name redaction.** Any object key matching `/^(access_token|refresh_token|id_token|client_secret|authorization|api_key|apikey|key|secret|password|signature|sig)$/i` has its value replaced, at any depth.
3. **URL handling.** Any string that parses as a URL has its query string removed when it contains a parameter matching rule 2 or the provider's known signing parameters (`x-expires`, `x-signature`, `refresh_token`, `_nc_*`, `oh`, `oe`); the path is kept so the payload still says which media it was. Paging URLs are dropped entirely and replaced with the bare cursor.
4. **Request records** store path and an allow-list of parameters (fields, metric, metric_type, period, breakdown, limit, since, cursor, pageToken, part, ids, dimensions, startDate, endDate). Nothing else.
5. **Assert before insert.** The ingest function re-runs a detector over the final JSON text (`/access_token|refresh_token|Bearer\s|client_secret|key=AIza/` plus every live secret value) and refuses the insert if anything matches. A refused insert fails the sync run with status failed and reason `sanitizer_leak`; it never degrades to storing.
6. **Versioned.** `sanitizer_version` is written on every payload row; a rule change bumps it.

Tests

- Fixture per location above, built from redacted real responses, asserting the exact output and that the injected fake token strings are absent from the serialized result.
- A property test: for random nesting depth and key placement, no injected secret survives.
- A leak-detector test that the assertion in rule 5 refuses a payload with a token in a paging URL, a header echo and a nested error.
- A boundary test that no adapter module imports `connectedAccounts`'s row type into an envelope.
- A scheduled audit query (the one used for this finding) as a script, runnable in CI against a scratch database and by hand against dev.

> **Remediation of existing rows (decision D7).** The 82 `raw_metrics` rows are display-layer data. Proposal: a one-off, marker-guarded script that rewrites `raw_metrics` in place through the same sanitizer, and rotates the affected Instagram token by forcing a refresh. The display projection does not read paging URLs or cover query strings, so nothing the athlete sees changes.

---

4 · Metric definitions

A metric registry in code (`server/services/intelligence/social/metrics.ts`) is the source of truth; `definition_version` on every observation points into it. Keys are platform-neutral names with a platform-availability matrix; a key present on one platform and absent on another is simply absent, never zero. Nothing is called "engagement rate".

Observed metrics (facts a provider states)

METRIC_KEY	ENTITY	UNIT	INSTAGRAM	TIKTOK	YOUTUBE	WINDOW
`followers`	account	count	followers_count	follower_count	absent (see subscribers)	point
`subscribers`	account	count	absent	absent	subscriberCount	point
`following · content_count · likes_total · views_total`	account	count	follows_count · media_count	following_count · video_count · likes_count	videoCount · viewCount	point

account_reach · account_views · profile_link_taps	account	count	insights reach / views / profile_links_taps	absent	absent	day (period=day)
views_30d · minutes_watched_30d · avg_view_pct_30d · subs_gained_30d · subs_lost_30d	account	count · count · percent · count · count	absent	absent	Analytics report	30d
like_count · comments_count	content	count	yes	yes (comment_count)	yes	point
share_count · save_count	content	count	shares · saved (insights)	share_count · absent	absent · absent	point
view_count	content	count	views (video/reel only)	view_count	viewCount	point
reach · total_interactions	content	count	insights	absent	absent	point
avg_watch_time_sec	content	seconds	ig_reels_avg_watch_time	absent	absent (channel-level only)	point
duration_sec	content	seconds	absent in Graph media (native when known)	duration	contentDetails.duration	static

Derived metrics (features, section 5), each with its denominator named

FEATURE_KEY	FORMULA	COMPARABLE ACROSS
engagement_per_follower	(like_count + comments_count) ÷ followers at the same observed_at, per content, then median over the window	Public-only proxy; IG and TikTok and YouTube each separately, never pooled
interactions_per_reach	total_interactions ÷ reach per content (IG connected only)	IG connected only
interactions_per_view	(like_count + comments_count + share_count) ÷ view_count per content	TikTok, YouTube, IG video separately
views_per_follower · reach_per_follower	view_count ÷ followers; reach ÷ followers (reach efficiency, §22.4)	Per platform
saves_per_1k_reach · shares_per_1k_reach · shares_per_1k_views	save_count × 1000 ÷ reach; share_count × 1000 ÷ reach; share_count × 1000 ÷ view_count	Per platform, per denominator
completion_pct	IG: avg_watch_time_sec ÷ duration_sec when both known; YouTube: avg_view_pct_30d (account-level)	Not comparable; different entities
story_reach_per_follower · story_completion	Once Stories are captured	IG only

5 · Feature calculator v1

Pure function: `calculateSocialFeatures({ observations, distributions, identities, asOf, windows })` → feature rows. It reads stored rows only, so a recompute for any past `as_of` is possible without a provider call (§28.10). Content-type segmentation uses `content_format`. Every feature carries `n` and lineage. Distinction: an **observation** is a provider fact with an `observed_at`; a **feature** is a number we computed from observations for a window ending at `as_of`.

FAMILY	FEATURES (PER PLATFORM UNLESS MARKED)	INPUTS
Follower level and growth	followers_latest · followers_growth_30d · followers_growth_90d (absolute and percent) · followers_growth_velocity (30d slope)	account followers or subscribers observations; growth needs ≥2 observations in the window else absent
Content performance distributions	views_median · views_p25 · views_p75 · views_p90 · reach_median · reach_p75 · reach_p90 · likes_median · comments_median, by content_format and overall	Latest observation per content within the window (posted_at inside the window)
Engagement distributions	engagement_per_follower_median · interactions_per_reach_median · interactions_per_view_median · saves_per_1k_reach_median · shares_per_1k_reach_median · shares_per_1k_views_median	Per-content ratios, then median; denominators as named in section 4
Distribution efficiency	reach_per_follower_median · views_per_follower_median · non_follower_reach_share (inferred, IG: share of posts whose reach ≥ 2× followers, replacing "is_discovery")	Marked inferred with evidence ids

Breakout and consistency	breakout_rate (share of posts with views or reach ≥ 3× the window median) · top_decile_share (share of total views from the top 10% of posts) · performance_cv (coefficient of variation of views or reach)	Same content set
Cadence	posts_per_week_30d · posts_per_week_90d · days_since_last_post · format_mix (share by content_format)	media_assets posted_at within the window, carousel frames excluded
Saves and shares	saves_median · shares_median · save_rate_median · share_rate_median	IG saved/shares; TikTok share_count
Story performance	story_reach_median · story_completion_median · story_reply_rate	Absent until Story capture exists; never zero
Audience	audience_share_top_country · audience_country_distribution (jsonb feature) · audience_gender_share · audience_age_distribution · audience_geo_concentration (HHI)	Latest distribution per dimension within the window, basis preserved
Cross-platform	effective_reach_cross_platform (sum of per-platform reach_median or views_median where present; platforms without a comparable metric are excluded and named in inputs) · platform_breadth	Per-platform features
Authority and confidence	Every feature: source_class = lowest authority used; confidence = f(n, authority, freshness); an explicit `coverage` input naming which metrics were absent	Section 6

> **Windows (decision D6).** Proposal: compute every distribution feature for 30 and 90 days, growth for 30 and 90, cadence for 30 and 90, with 7-day variants only for cadence and follower growth. as_of defaults to the latest observation; the Mia command takes it as an argument.

---

6 · Source precedence at the metric and window level

1. For each (identity, entity, metric_key, window) the resolver collects every observation whose observed_at falls in the window, grouped by source_class.
2. Authority: `connected_verified > platform_public > web_researched > imputed`. The highest class with at least one fresh observation (freshness per metric from the `research_freshness` family, social_public 7 days, social_connected 2 days) wins the window; lower-class observations are ignored for calculation but stay in the table.
3. Freshness does not fall through silently: a stale connected observation with a fresh public one yields the public value and a lower source_class on the feature, and the resolution is recorded in `inputs`.
4. `inferred` never competes with observations. It is only ever a feature (non_follower_reach_share, content_format when the provider does not state it) and its row carries the evidence ids it was derived from.
5. Unknown stays unknown: if no observation of any class exists in the window, the feature row is not written. There is no zero. Step 4's priors fill the gap and set `imputed`.
6. Superseded observations are never deleted or flagged; supersession is a read-time decision that depends on the window and the freshness parameters, so it must be recomputable.

This is the same rule `refreshScores.pickBestPerPlatform` applies today at the level of a whole platform row (connected 4 > provider 3 > discovery 2 > research 1, 45-day freshness), moved to the metric level and made explicit in lineage. The Mia validation compares the two.

---

7 · The display analytics contract

`social_analytics_snapshots`, `SocialAnalyticsResult`, the four `/analytics` routes, Mila's `get_analytics`, the deck's audience card and the score refresh's `buildSocialFromAnalytics` keep reading exactly what they read today. The projection that fills the daily row is computed from the native envelope by a function pinned to today's output with fixture tests, so the row is byte-identical to what the old code produced. The only new statement about it is a comment: it is a projection and stops being a scoring input in Step 4, when `refreshScores` reads `social_features`. No route, client or Mila change in Step 1.

8 · media_assets: reuse or a new content table

What `media_assets` is today: the unified asset library (spec: docs/durable-media-rehosting.md), one row per owned pixel set, with content identity (`athlete_id` , `platform` , `source_media_id` , unique), content facts (permalink, caption, posted_at, media_type incl. CAROUSEL_CHILD frames, parent), mutable counters (like_count, comments_count, insights, refreshed on every re-see), pixel lifecycle (gcs paths, status stored | failed | skipped_size), enrichment (metadata.vision, embedding, pixel_grade), and product lifecycle (origin connected | discovery | upload | generated, deal_id, approval_status, deleted_at, generation_job_id). 569 rows for Mia; 561 connected, 21 uploads, 169 generated across all athletes; full backfill complete for her three platforms.

	REUSE MEDIA_ASSETS (RECOMMENDED)	NEW SOCIAL_CONTENT TABLE
Content identity	Already unique on (athlete, platform, source_media_id); carousel frames modelled	Would duplicate the same key and need a back-reference to media_assets for pixels and enrichment
Platform-native fields	Add `native jsonb` + `content_format` + `native_version` ; TikTok title/description/duration, IG media_product_type/is_shared_to_feed, YouTube duration/topics stop being collapsed into `caption`	Clean by construction, but every consumer (Curator, kit, Studio, weekly spine, first look) reads media_assets and would need a join or a sync
Mutable counters	Rule change: `like_count` , `comments_count` , `insights` become display projections; history and scoring inputs live in observations. No column removed	Same rule, one table further away
Non-social rows in the same table	origin upload | generated rows are excluded by `origin` in every social query; already the pattern (studio/content.ts)	Avoided
Pixel-status coupling	A content item whose pixels failed still has a row (status failed); the observation layer does not care about pixels. Stories, which have no durable pixels, get observations only (or a row with content_format story and no gcs path, decision D4)	Cleaner for Stories and for content we never re-host
Ownership and deletion	Cascade on athlete delete; soft delete via deleted_at; observations keep asset_id set null on hard delete	Same
Migration cost	Three nullable columns, one FK, a backfill of native from raw_metrics.recent_media where present	New table plus a backfill of 569+ rows and a lineage column on media_assets anyway

> **Recommendation.** Reuse, with the three additive columns, the counter rule, and the explicit convention that `media_assets` is the durable content entity for social platforms while pixels and product lifecycle remain its other roles. The one place reuse strains is Stories (no pixels); decision D4 covers it. Bring back if you would rather pay the duplication for a pure content table now.

9 · The Mia validation fixture

One command, deterministic, no network:

```
npx tsx --require dotenv/config server/scripts/socialFeatureVector.ts --athlete dc8f37f1-… --as-of 2026-09-09T00:00:00Z --out ./out/mia-2026-09-09.json
# or, without a database, from committed redacted fixtures:
npx tsx server/scripts/socialFeatureVector.ts --fixtures server/services/intelligence/social/__fixtures__/mia --as-of 2026-09-09T00:00:00Z
```

Output is one JSON document with three sections and a lineage index: **raw** (the sanitized payloads used, by sync run and page, with hashes), **observations** (every row that fell inside any window, grouped by entity and metric, with source_class and observed_at), **features** (every feature row with value, n, window, source_class, confidence, calculation_version and the observation ids it used). A companion `--explain followers_growth_30d` prints the resolution for one feature: which observations were candidates, which class won, why the others were ignored. The jest suite runs the same calculator over the committed fixtures and asserts exact values for a hand-checked subset (medians, p75, cadence, growth), so a change in the calculator is a visible diff.

Where the fixture data comes from. Mia's 72 Instagram, 73 TikTok and 6 YouTube daily rows (30 Jun to 10 Sep) hold account facts, account insights, 18 to 20 recent media with counts per day, and demographics; her 569 media rows hold posted_at, formats, counts and enriched insights for 34

videos. The backfill (section 10) turns these into observations with source_method `legacy_snapshot` and `legacy_media_assets`; redacted copies of a few days plus one native envelope captured on the first flagged sync become the committed fixtures. Her YouTube connection is expired, which the command will report as coverage, not as zero (§29.1).

---

10 · Rollout

Migrations

- **0044_social_substrate.sql**: the six tables with their CHECKs, unique constraints and indexes; the three columns, FK and index on media_assets. Pure DDL, idempotent, listed in `db-migrate.sh`. Applied only through the deploy path.
- Parameter rows (drafts via `seedModelParams.ts`, family `signals`): metric registry version, sanitizer version, feature windows, freshness values already in `research_freshness`.
- Migration numbering: 0044 next; PR #309's 0034 remains its own problem.

Dual write, behind a flag

- `SOCIAL_SUBSTRATE_WRITE` (default off in both YAMLs). Off: the code path is the old one, byte for byte. On: `collectSocialAnalytics`, `collectDiscoveryAnalytics`, the backfill tick and `enrichTopPostInsights` run sanitize → store → adapt → features after their existing writes; a failure in the new path is logged on the sync run and never fails the old write.
- No reader of the new tables in production; the score, the Growth tab and Mila are unaffected by construction.

Backfill

1. `backfillSocialIdentities.ts`: from connected_accounts (active, expired and revoked, so history is preserved) and athletes.socials; marker-guarded, idempotent.
2. `backfillSocialObservations.ts`: walks social_analytics_snapshots (218 rows) and media_assets (~750 social rows) into observations and distributions with source_method legacy_*, observed_at = snapshot_date or metadata timestamps, definition_version legacy/1. Sanitizes raw_metrics on the way (decision D7 decides whether it also rewrites the source rows). Idempotent on the unique constraints.
3. `socialFeatureVector.ts` for Mia at three as_of dates as the acceptance check.

Verification

- Projection fixture tests: the display row produced from the native envelope equals today's output for Instagram, TikTok and YouTube fixtures.
- Flag off: the byte-for-byte check from Step 0 repeated (dry-run score refresh and athletes_current export).
- Flag on in dev for 3 nightly runs: sync runs status ok, zero sanitizer refusals, observation counts match items fetched, the audit query returns 0 rows for the new tables.
- Mia feature vector at as_of = 9 Sep compared against the current display row and against `buildSocialFromAnalytics`'s resolution; differences classified as measurement improvement, definition mismatch or missing data (§23).

Rollback

- Flag off stops all writes; nothing reads the tables, so rollback is a config flip. Tables and columns stay (dropping is a later, separate migration).
- media_assets columns are nullable and unread by existing code; ingestOne's extra writes are inside the flag.

Storage and cost

TABLE	ESTIMATE	BASIS
raw_source_payloads	~50 KB per Instagram daily sync, ~20 KB TikTok, ~5 KB YouTube; ≈ 30 MB per athlete-year at today's cadence before retention	Measured raw_metrics sizes (50,028 / 19,410 / 280 bytes average)
social_metric_observations	≈ 25 content × 6 metrics + 10 account metrics ≈ 160 rows per platform per day ≈ 60k rows per athlete-year; tens of bytes each	Current recent-media size 18 to 20 per day
social_features	≈ 120 rows per athlete per computation; one per night	Feature list × windows × platforms

Provider calls	No new calls in Step 1 except Stories if enabled; discovery pagination increases are Step 1b	Same fetches, second destination

11 · Decisions needed before implementation

#	DECISION	OPTIONS	RECOMMENDATION
D1	Durable content entity	Reuse media_assets with three additive columns and the counter rule; or a new social_content table with a lineage link	Reuse (section 8)
D2	Raw payload retention	Forever; 180 days; 90 days; 30 days. Observations survive deletion either way	180 days for connected, 90 for public, as a versioned parameter; revisit after the first cost report
D3	Full payloads or selected envelopes	Full sanitized response envelope per page; or a curated subset per endpoint family	Full envelope. The spec's acceptance test is "recompute without refetching", and a subset chosen today is exactly the "fields intentionally not collected" problem in reverse
D4	Story capture	Not in Step 1; every 6 hours; every 12 hours (a Story lives 24 h); and whether a Story gets a media_assets row (cover only) or observations only	Every 8 hours behind its own flag, observations only, in a Step 1b PR after the base lands
D5	Historical observations after OAuth disconnect or revoke	Keep everything, identity stays active with connected_account_id nulled; keep but mark the identity superseded; delete on athlete request only	Keep; identity status unchanged (the account is still hers); freshness alone makes old connected observations lose to newer public ones; deletion only through account deletion
D6	Initial feature windows	7/30/90; 30/90 only; 30/90/365	30 and 90 for everything, 7 additionally for cadence and follower growth
D7	Scrub existing raw_metrics rows and rotate the exposed Instagram token	Rewrite the 82 rows in place through the sanitizer; or leave display history untouched and only stop the bleeding	Rewrite and rotate. They are display rows; nothing reads the paging URLs
D8	Flag scope	One flag for the whole substrate; or per-platform flags	One flag; per-platform failures are isolated by sync run status

12 · Access model check (ADR-015, ADR-016)

Recorded after the design was written, against the two LOCKED decisions: a user is not an athlete, and affiliation does not imply access to private data. Nothing below changes Step 1 scope; it says where the proposal and the current code stand.

ITEM	STATUS	NOTE
Domain owner of every new table	COMPLIANT	social_identities, sync runs, payloads, observations, distributions and features are owned by `athlete_id`, never by an auth user. No `organization_id` anywhere.
Consent linkage	COMPLIANT	Connected data is tied to the athlete's own `connected_accounts` row through `social_identities.connected_account_id` and `source_method`; that row is the consent record a future authorizer reads.
Private versus public separability	COMPLIANT	`source_class` on every observation, distribution and feature distinguishes connected_verified (private, consented) from platform_public and web_researched (public). A feature's class is the lowest authority of its inputs, so a number derived from private data is never public by default.
media_assets reuse	FLAG	The table already mixes consented uploads (`origin = upload`, camera roll) with public-facing posts. Reuse does not worsen this, but `origin` becomes the access-relevant field for content and must be kept accurate; a Story row, if created, would be private content.
Current code: athletes doubles as the user table	FLAG	`athletes.auth_id`, `account_roles → athletes.id`, and `requireAuth` setting `req.athleteId` all assume athlete = user. Step 1 does not touch this. Step 3 (identity confirmation) must link a user to an athlete rather than treating the signup row as the athlete; that is the first place ADR-015 changes a design.
Backfill of identities from connected_accounts	COMPLIANT	Identity rows are keyed by the platform account id, not by who logged in; a second user later acting for the same athlete needs no change here.

What the implementation PR would contain, once approved

- 0044 migration + schema.ts; metric registry + sanitizer + ingest + three adapters + projection functions + feature calculator + resolver; flag wiring in the four skills; two backfill scripts and the Mia command; fixtures and tests (sanitizer, projections byte-for-byte, adapters, calculator, precedence, boundary); build record at `docs/architecture/phase-a/step1-social-substrate.md`.
- Likely two PRs: 1a the substrate and flag (no Stories), 1b Stories capture and the discovery pagination expansion (§26.2), so the base can be verified in shadow first.